Image Grayscale Conversion Using SIMD-Based AVX2 Assembly Language

CEPARCO — Integrating Project
Abdulrahman, Lim, Lopez, Nicolas
Video: CEPARCO-G7.mp4

0. Abstract

Image processing involves performing mathematical operations on large amounts of pixel data. One common operation is ***grayscale conversion***, which transforms colored images into shades of gray using the red, green, and blue components of each pixel. Though the conversion process is relatively simple, processing high-resolution images requires performing the same calculations repeatedly across thousands or millions of pixels.

This project proposes the implementation and performance evaluation of image grayscale conversion using sequential C programming and SIMD-based AVX2 assembly language. This aims to demonstrate how data-level parallelism can be used to accelerate repetitive image processing operations. The sequential C implementation will serve as the reference, while the SIMD implementation will use 256-bit YMM registers to process multiple pixel values simultaneously.

The proposed system will accept colored BMP images of different resolutions, convert them into grayscale, and measure the execution times of both implementations. The project will also evaluate output correctness, speedup, and how image resolution affects processing performance. Moreover, the study will examine whether the performance improvements achieved through SIMD computation remain significant when accounting for memory access and data preparation.

Through these comparisons, the project aims to demonstrate the advantages and limitations of SIMD-based parallel computing in a practical image processing application.

1. Description of the Proposed Project

Digital image processing is widely used in photography, document scanning, computer vision, and multimedia applications. These applications often involve manipulating large amounts of image data through operations like filtering, brightness adjustment, and color conversion. One of the most basic image processing operations is grayscale conversion. This process converts a colored image into a representation containing different shades of gray while preserving the relative brightness of its pixels.

In a sequential implementation, the program processes pixels one after another. While each pixel only requires a few arithmetic operations, the total number of calculations increases with the resolution of the image. For example, a 1920 × 1080 image contains more than two million pixels, each requiring its own grayscale calculation. Since each pixel can be processed independently, grayscale conversion is suitable for Single Instruction, Multiple Data (SIMD) parallelism. SIMD allows one instruction to perform an operation on multiple data values simultaneously.

This project will investigate the performance of grayscale conversion implemented using C and x86-64 AVX2 assembly language. Rather than developing a new grayscale conversion formula, the project will focus on understanding how SIMD instructions, memory organization, and image resolution affect execution performance.

The main objective is to determine whether a manually implemented SIMD-based grayscale conversion kernel can achieve better performance than a sequential C implementation.

The specific objectives are:

1. To implement a working grayscale conversion program using C.
2. To develop a SIMD-based grayscale conversion kernel using AVX2 assembly language.
3. To verify that both implementations produce equivalent grayscale images.
4. To measure and compare execution times across different image resolutions.
5. To analyze the resulting performance improvements and identify factors that limit SIMD speedup.

A. Inputs of the Project

The program will accept colored digital images containing red, green, and blue (RGB) color values. The proposed inputs include:

- RGB images of various resolutions.
- Pixel data that contains red, green, and blue components.
- The image width and height, which determine the total number of pixels.

The group plans to use uncompressed BMP images to simplify reading and processing image data using C. Different image resolutions will be tested to observe how increasing image size affects execution time.

B. Proposed Process

The proposed system will convert RGB images into grayscale by applying a weighted calculation to the red, green, and blue components of each pixel.

The grayscale conversion formula is:

$$\mathbf{Gray = 0.299(R) + 0.587(G) + 0.114(B)}$$

Where R, G, and B represent the red, green, and blue color values, respectively. The formula assigns different weights to each color component. Green receives the largest weight, followed by red and blue, reflecting their relative contributions to perceived brightness.

The program will perform the following processes

Step 1: Image Loading

The C program will read the BMP file, identify the image dimensions, and obtain the pixel data.

Step 2: Pixel Data Preparation

The RGB values will be extracted and arranged into separate arrays suitable for processing.

This arrangement will allow the SIMD implementation to load groups of color values more efficiently.

Step 3: Grayscale Calculation

The program will calculate the grayscale intensity of each pixel using the weighted formula. Two versions of this calculation will be implemented: sequential C and SIMD-based assembly.

Step 4: Output Generation

The calculated grayscale values will be converted to valid pixel intensities and written to an output BMP image.

Step 5: Correctness Verification

The grayscale values produced by the sequential and SIMD implementations will be compared to determine whether both versions produce equivalent results.

Step 6: Performance Measurement

The execution time of each implementation will be measured and compared across different input resolutions.

i. Processes to Be Parallelized Using SIMD

The primary process to be parallelized is the grayscale calculation. In the sequential C implementation, the program will process each pixel individually using a loop. For every pixel, the program will read the RGB components, multiply them by their respective weights, add the results, and store the grayscale intensity.

In the SIMD implementation, the same arithmetic operations will be performed on multiple pixels simultaneously. The project will use 256-bit YMM registers, which can hold eight 32-bit floating-point values. This allows the SIMD kernel to calculate grayscale intensities for eight pixels per vector iteration.

The proposed SIMD operations include:

- Loading groups of pixel component values into YMM registers.
- Multiplying the RGB components by their corresponding grayscale weights.
- Adding the weighted components to calculate grayscale intensity.
- Storing the resulting grayscale values in memory.

By reducing the number of loop iterations and performing multiple arithmetic operations in parallel, the SIMD implementation is expected to reduce the computation time required for grayscale conversion. However, the actual speedup will depend on factors such as memory access, instruction overhead, and compiler optimization.

ii. Existing Implementations and Related Works

Grayscale conversion is a well-established image processing operation. Existing image processing libraries, such as OpenCV, provide functions for converting colored images into grayscale. OpenCV supports color-space conversion through functions such as `cvtColor`, which can perform grayscale conversion without requiring users to manually implement the underlying calculations.

Another relevant implementation is the Simd Library, which provides optimized image processing operations using instruction sets such as SSE and AVX. This demonstrates how SIMD instructions can be applied to accelerate operations involving large amounts of pixel data. These implementations show that grayscale conversion and SIMD optimization are existing techniques.

The proposed project will differ primarily in its experimental focus. Instead of relying on existing image processing libraries to perform grayscale conversion, the group will manually implement the conversion using C and NASM assembly language. The group will examine the performance of the grayscale calculation itself and evaluate the overhead introduced by loading, arranging, and storing image data. Its contribution will then focus on the implementation, benchmarking methodology, and analysis of sequential and SIMD execution, rather than claiming to introduce a new grayscale conversion algorithm.

iii. Plans for Parallelizing the Proposed Process

The project will be developed in several stages.

Stage 1: Sequential Implementation

The group will first implement the grayscale conversion algorithm in C. The program will read RGB image data, calculate the grayscale intensity of each pixel, and save the resulting image. This version will serve as the reference for correctness and execution-time comparisons.

Stage 2: Data Preparation

The RGB components will be stored in separate arrays of single-precision floating-point values.

This arrangement will simplify the loading of values into SIMD registers, allowing the group to focus on implementing the vector arithmetic operations.

Stage 3: SIMD Implementation

The group will implement the grayscale calculation in x86-64 assembly using AVX2-compatible instructions and YMM registers.

The assembly kernel will process eight pixels per vector iteration. Any remaining pixels that cannot form a complete group of eight will be processed separately.

Stage 4: Correctness Testing

Both implementations will process the same images. The group will compare their calculated grayscale values using an appropriate numerical tolerance to account for possible floating-point differences. The final grayscale images will also be checked to ensure that the conversion produces valid pixel values.

Stage 5: Performance Evaluation

The group will measure the execution time of the grayscale calculation using a high-resolution timer. Each implementation will be executed repeatedly to reduce the effect of timing variations. Average execution times will be calculated for each image resolution. The primary measurements will focus on the grayscale kernels. A separate comparison of total processing time may also be performed to evaluate the overhead caused by data preparation and image handling.

Testing will be performed using optimized Release builds. The group will also document the compiler optimization settings to ensure that performance results are interpreted correctly.

iv. Processes That Will Not Be Parallelized

The following processes will initially remain sequential:

1. Reading input images from BMP files.
2. Obtaining the image dimensions and pixel information.
3. Allocating memory for input and output arrays.
4. Preparing and arranging pixel data.
5. Writing the resulting grayscale image.
6. Measuring and displaying execution times.

7. Comparing the outputs of both implementations.

These operations will be handled by the main C program. The SIMD assembly module will focus primarily on the grayscale calculation, allowing the performance comparison to isolate the effect of vectorized computation.

C. Outputs of the Project

The expected outputs include:

1. Grayscale BMP images generated from the original colored inputs.
2. A functional sequential C implementation of grayscale conversion.
3. A functional AVX2 assembly implementation of grayscale conversion.
4. Correctness verification results comparing both implementations.
5. Execution-time measurements for different image resolutions.
6. A comparison table showing the average execution time of each implementation.
7. Calculated speedup values.
8. An analysis of the factors affecting SIMD performance.

The speedup will be calculated using:

Speedup = Sequential Execution Time / SIMD Execution Time

A speedup greater than 1 indicates that the SIMD implementation is faster than the sequential implementation for the measured workload.

The project will also examine whether increasing image resolution produces proportional improvements in SIMD performance.Possible limiting factors include memory bandwidth, data preparation overhead, and additional instructions required for SIMD processing.

2. Proposed Target Implementation Platform

Component	Proposed Platform
Programming language	C
Parallel implementation	x86-64 SIMD assembly
Instruction set	AVX2-compatible instructions
SIMD registers	256-bit YMM registers
Assembler	NASM
Development environment	Microsoft Visual Studio
Operating system	Windows 64-bit
Processor	x86-64 CPU with AVX2 support
Image format	24-bit uncompressed BMP
Version control	GitHub

The main C program will handle image input/output, memory allocation, data preparation, performance measurement, and result verification. The x86-64 assembly module will contain the SIMD-based grayscale calculation. The project will be compiled as a 64-bit application to support the required registers and calling conventions.

3. Mid-Project Milestones

Milestone 1 — Week 6: Project Proposal

The group will finalize and submit the project proposal, including the problem description, objectives, proposed algorithm, SIMD implementation plan, target platform, expected outputs, and related works.

Milestone 2 — Week 10: Mid-Project Update and Demo

The group aims to complete the following:

1. Set up the Visual Studio and NASM development environment.
2. Implement and test BMP image reading and writing.
3. Complete the sequential C grayscale conversion program.
4. Generate correct grayscale images from sample inputs.
5. Prepare the pixel arrays required for SIMD processing.
6. Develop an initial AVX2 assembly kernel capable of processing sample pixel data.
7. Present the working C version and demonstrate preliminary SIMD integration.

Milestone 3 — Week 13: Complete Project Submission and Demo

The group aims to complete the following:

1. Finalize the SIMD-based AVX2 grayscale conversion implementation.
2. Integrate the C and assembly modules.
3. Verify the correctness of both implementations.
4. Conduct performance tests using multiple image resolutions.
5. Compare average execution times and calculate speedup.
6. Analyze factors affecting performance and identify implementation limitations.
7. Prepare the final demonstration and project documentation.
8. Upload the complete project files and results to GitHub.

4. References

[1] OpenCV, "Color Conversions," *OpenCV Documentation*. https://docs.opencv.org/4.10.0/de/d25/imgproc_color_conversions.html

[2] Intel Corporation, "Intel Intrinsics Guide." https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html

[3] Simd Library, "Simd Image Processing Library." GitHub. https://github.com/ermig1979/Simd

[4] Intel Corporation, "Intel 64 and IA-32 Architectures Software Developer's Manuals." https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html